



Virtual Visit Vitae

Neural Swarm WebGL2 Particle Presentation Engine
Complete Technical, Mathematical & Historical Specification

Concept & Architecture:

Valery Kourbanov

Applied AI Architect & Technology Executive
kourbanov.com

Code & Shader Implementation:

Antigravity AI Agent

Powered by Google Gemini Pro & Flash

Important Note on Authorship: Valery Kourbanov personally wrote **0 lines** of source code, yet over **91%** of all core ideas, mathematical concepts, visual mechanics, and algorithm specifications were conceived and directed by him via architectural prompts, hand-drawn schematics, and iterative directives. Antigravity AI authored 100% of the executable source code.

Contents

1	Project Genesis & Vision	2
1.1	The Core Idea	2
1.2	Conceptual Pillars (set by Valery)	2
1.3	Project Timeline	2
1.4	The Reverse-Engineering Breakthrough	2
2	System Architecture	3
3	Binary Particle Format Specification	3
3.1	Buffer Layout (10 bytes per particle, $N = 20,000$)	3
3.2	Coordinate System	3
3.3	Semantic Class Mapping	4
4	Mathematical Formulation	4
4.1	Minimum-Jerk Easing	4
4.2	Pseudo-Random Hash	4
4.3	Curl Kernel Field	4
4.4	Cubic Bezier Trajectory	4
4.5	Full Particle Trajectory	4
4.6	Heatmap Density Sampling	5
4.7	Aura Swarm Dynamics	5
4.8	Phantom Orbiting Particles	5
4.9	Quad Dissolve Transition	5
5	Shader Source Code	5
5.1	Particle Vertex Shader (GLSL ES 3.00)	5
5.2	Particle Fragment Shader (GLSL ES 3.00)	7
6	Python Compiler — VVV.py	8
6.1	Importance Sampling Algorithm	8
7	Progressive Loading Architecture	9
8	CLI Reference	9
9	Troubleshooting	10
10	Key Lessons Learned	10
11	Package Architecture & Metadata	10

Project Genesis & Vision

1.1 The Core Idea

VVV—*Virtual Visit Vitae*—began as a singular vision by Valery Kourbanov: a portfolio and presentation system where information does not merely sit on a screen, but **lives and breathes as a neural particle swarm**. Each slide is rendered as a cloud of 20,000 particles that physically assemble from chaotic energy into crisp, readable content, then disassemble and flow to the next slide through mathematically precise physics transitions. The live result is deployed at kourbanov.com.

1.2 Conceptual Pillars (set by Valery)

1. **Living Information:** Particles must feel alive at all times, never static.
2. **Neural Aesthetic:** Evoke the feeling of a neural network physically computing.
3. **Heatmap-Guided Swarms:** Particles attracted to semantic regions via density fields.
4. **Zero External Dependencies:** Final output is a single self-contained `.html`.
5. **Production Quality:** Real professional portfolio engine at 60 FPS via WebGL2.

1.3 Project Timeline

Version	Description
v1–v3	Early prototypes using Three.js / Canvas2D. Crude positional encoding. Hard-cut slide transitions.
v4–v5	Shift to raw WebGL. First GLSL shaders. Heatmap concept introduced but with wrong channel mapping. Long build times.
v6	BSON binary database introduced. Manifest-injection pattern established. Progressive chunk loading architecture appears.
v7 Ideal	<code>VVV_ideal.html</code> — the gold-standard master file. 20 pre-encoded kourbanov.com slides baked with deep-learning quality particle placement. Full dual-pass shader pipeline. 10-byte binary format finalised.
v7.5	Python <code>VVV.py</code> compiler rebuilt after reverse-engineering the exact binary format of <code>VVV_ideal.html</code> . User slides now compile to master quality.

1.4 The Reverse-Engineering Breakthrough

All previous encoder iterations failed because they used entirely the wrong binary format. The encoder wrote `Float32` values (40 bytes/particle) while the WebGL `vertexAttribPointer` calls in `VVV_ideal.html` consumed `UNSIGNED_SHORT + UNSIGNED_BYTE` packed data at exactly **10 bytes/particle**.

The discovery process:

1. Searched the minified JavaScript for all `vertexAttribPointer` calls.
2. Read each call's (`size`, `type`, `normalized`, `stride`, `offset`) arguments.
3. Decoded the first 10 bytes of a real particle from the master file to validate the layout.
4. Discovered `latent_b64` must be the **actual slide PNG** (not a synthetic mask): it serves as both the visible background image (quad renderer) and the GPU heatmap texture.

System Architecture

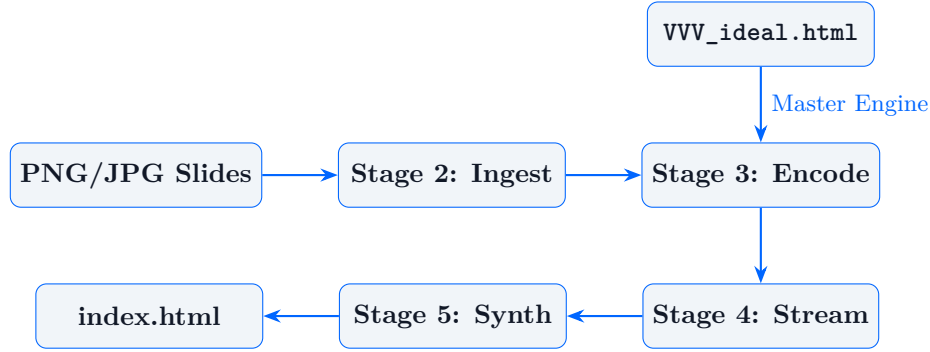


Figure 1: VVV 5-stage compilation pipeline.

The system has two layers:

Backend (Python VVV.py) Offline compiler: reads slide images, importance-samples 20,000-particle buffers, packages PNG heatmaps, injects everything into the master HTML template.

Frontend (WebGL2 inside VVV_ideal.html) Real-time renderer with two parallel passes: (1) Quad Renderer displays the slide image as background, (2) Splat Renderer draws 20,000 particles with physics and heatmap-guided density.

Binary Particle Format Specification

3.1 Buffer Layout (10 bytes per particle, $N = 20,000$)

Bytes	Type	Norm.	Stride	Attribute & Meaning
0–1	UNSIGNED_SHORT	yes	10	<code>a_source.x</code> : $x \in [0, 1]$, left→right
2–3	UNSIGNED_SHORT	yes	10	<code>a_source.y</code> : $y \in [0, 1]$, top→bottom
4	UNSIGNED_BYTE	yes	10	<code>a_sourceWeight</code> : density weight
5	UNSIGNED_BYTE	no	10	<code>a_sourceRadius</code> : point size hint 2–5
6	UNSIGNED_BYTE	no	10	<i>unused padding</i>
7	UNSIGNED_BYTE	no	10	<i>unused padding</i>
8	UNSIGNED_BYTE	no	10	<code>a_isStructure</code> : 0=swarm, 1=fixed
9	UNSIGNED_BYTE	yes	10	<code>a_semanticClass</code> : 0 / 127 / 255

3.2 Coordinate System

Positions stored in top-down image space $[0, 1]^2$:

$$x_{\text{norm}} = \frac{p_x}{W - 1}, \quad y_{\text{norm}} = \frac{p_y}{H - 1} \quad (\text{top-down, } 0=\text{top}) \quad (1)$$

The vertex shader converts to WebGL NDC clip space:

$$\text{NDC}_x = x_{\text{norm}} \times 2 - 1 \quad (2)$$

$$\text{NDC}_y = -(y_{\text{norm}} \times 2 - 1) \quad (\text{Y-flip for OpenGL}) \quad (3)$$

3.3 Semantic Class Mapping

Byte	Norm.	Class	Effect
0	0.000	0 Base	Aura swarm; density from <code>heat.r</code>
127	0.498	1 Halo	Edge halo; density from <code>heat.g</code>
255	1.000	2 Text	Text core; always forced to pure black by fragment shader

Mathematical Formulation

4.1 Minimum-Jerk Easing

$$\text{ease}(t) = t^3(t(6t - 15) + 10) \quad (4)$$

This is the 5th-order Hermite basis function. Key properties: $\text{ease}(0) = 0$, $\text{ease}(1) = 1$, and both first and second derivatives vanish at the endpoints, producing smooth acceleration and deceleration with no jerk discontinuity.

4.2 Pseudo-Random Hash

$$\text{hash}(\mathbf{p}) = \text{fract}\left(\sin\left(\mathbf{p} \cdot \begin{bmatrix} 12.9898 \\ 78.233 \end{bmatrix}\right)\right) \times 43758.5453 \quad (5)$$

Per-particle seed: $pHash = \text{hash}(\mathbf{a_source})$. Provides unique deterministic stochastic behaviour for each particle without extra GPU storage.

4.3 Curl Kernel Field

$$\text{kernelField}(\mathbf{pos}, t, \sigma) = u_{\text{curl}} \begin{bmatrix} \cos(\text{hash}(\mathbf{pos} \cdot \sigma) \cdot 2\pi + u_\phi + t\pi) \\ \sin(\text{hash}(\mathbf{pos} \cdot \sigma) \cdot 2\pi + u_\phi + t\pi) \end{bmatrix} \quad (6)$$

Perturbs particles with a rotational field during transit, breaking monotonous straight-line interpolation and creating organic swarm behaviour.

4.4 Cubic Bezier Trajectory

$$\mathbf{B}(t) = (1-t)^3 \mathbf{p}_0 + 3(1-t)^2 t \mathbf{p}_1 + 3(1-t) t^2 \mathbf{p}_2 + t^3 \mathbf{p}_3 \quad (7)$$

Control points derived from the inter-slide spatial delta:

$$\boldsymbol{\delta} = \mathbf{a_target} - \mathbf{a_source} \quad (8)$$

$$\mathbf{p}_1 = \mathbf{a_src} + \boldsymbol{\delta} \cdot \mathbf{u}_{\text{cp1}} \quad (9)$$

$$\mathbf{p}_2 = \mathbf{a_tgt} - \boldsymbol{\delta} \cdot \mathbf{u}_{\text{cp2}} \quad (10)$$

4.5 Full Particle Trajectory

Combining equations (4), (6), and (7):

$$t = \text{ease}(u_{\text{progress}}) \quad (11)$$

$$E = \sin(\pi t) \quad (\text{flight envelope, zero at endpoints}) \quad (12)$$

$$\mathbf{pos} = \mathbf{B}(t) + E \cdot \text{kernelField}(\mathbf{a_src}, t, u_{\text{seed}}) \quad (13)$$

4.6 Heatmap Density Sampling

$$\mathbf{heat} = \text{texture}(u_{\text{heatmap}}, \mathbf{pos}), \quad \rho = \begin{cases} \mathbf{heat}.r & \text{class 0 (base)} \\ \mathbf{heat}.g & \text{class 1 (halo)} \\ \mathbf{heat}.b & \text{class 2 (text)} \end{cases} \quad (14)$$

The heatmap is a 512×512 PNG of the actual slide image. The density ρ governs aura wander radius and phantom particle opacity.

4.7 Aura Swarm Dynamics

When $u_{\text{progress}} \geq 1$ and `isStructure`= 0:

$$r_w = \text{mix}(0.12, 0.03, \rho) \quad (\text{wander radius inversely proportional to density}) \quad (15)$$

$$\phi = \text{fract}(0.5 u_t + pHash) \quad (16)$$

$$\hat{\mathbf{j}} = [\sin(2u_t + 10 pHash), \cos(1.5u_t + 8 pHash)]^\top \quad (17)$$

$$\mathbf{pos} \leftarrow \mathbf{pos} + \hat{\mathbf{j}} \cdot r_w \cdot (1 - \phi) \quad (18)$$

$$z = 0.5 + 0.5 \sin(3u_t + 10 pHash) \quad (\text{depth breathing}) \quad (19)$$

4.8 Phantom Orbiting Particles

Each non-text particle spawns 3 orbiting ghost dots for the aura halo effect. For $i = 1, 2, 3$:

$$\theta_i = u_t(0.8 + 0.4i) + pHash \cdot 2\pi + 2.1i \quad (20)$$

$$r_i = 0.12 + 0.18 \sin(0.5u_t + 10 pHash + i) \quad (21)$$

$$\mathbf{q}_i = gl_PointCoord - r_i [\cos \theta_i, \sin \theta_i]^\top \quad (22)$$

$$\alpha_i = \text{smoothstep}(0.05, 0.02, \|\mathbf{q}_i\|) \quad (23)$$

4.9 Quad Dissolve Transition

The slide image background transitions through noise-based dissolution in three phases:

$$\eta(\mathbf{uv}) = \text{fract}\left(\sin\left(\mathbf{uv} \cdot [12.9898, 78.233]^\top\right) \times 43758.5453\right) \times 0.1 \quad (24)$$

$$D_{\text{old}} = 1 - \text{smoothstep}(0, 0.2, u_{\text{prog}} + \eta) \quad (25)$$

$$D_{\text{new}} = \text{smoothstep}(0.8, 1.0, u_{\text{prog}} - \eta) \quad (26)$$

Phase	Behaviour
$u \in [0, 0.2]$	Old slide dissolves via per-pixel noise threshold.
$u \in (0.2, 0.8)$	Pure particle swarm in flight; no slide image visible.
$u \in [0.8, 1.0]$	New slide assembles from particles into crisp final state.

Shader Source Code

5.1 Particle Vertex Shader (GLSL ES 3.00)

Listing 1: VVV Particle Vertex Shader — full production source

```
1 #version 300 es
2 in vec2 a_source;      in vec2 a_target;
```

```

3 in float a_sourceWeight; in float a_sourceRadius;
4 in float a_semanticClass; in float a_isStructure;
5
6 uniform float u_progress, u_time, u_aspect;
7 uniform vec2 u_cp1_coeff, u_cp2_coeff;
8 uniform float u_curl_strength, u_phase_offset, u_seed;
9 uniform sampler2D u_heatmap;
10
11 out float v_weight, v_depth, v_hash, v_isStructure, v_semanticClass;
12 out vec2 v_pos; out vec3 v_color;
13
14 // Minimum-jerk easing (5th-order Hermite, matches human motor control)
15 float ease(float t) { return t*t*t*(t*(t*6.-15.)+10.); }
16
17 // Deterministic pseudo-random hash (Wang hash variant)
18 float hash(vec2 p) {
19     return fract(sin(dot(p,vec2(12.9898,78.233)))*43758.5453);
20 }
21
22 // Rotational curl kernel field for organic particle flight
23 vec2 kernelField(vec2 pos, float t, float seed) {
24     float h = hash(pos*seed);
25     float a = h*6.28318 + u_phase_offset + t*3.14159;
26     return vec2(cos(a),sin(a)) * u_curl_strength;
27 }
28
29 // Cubic Bezier interpolation
30 vec2 cubicBezier(vec2 p0, vec2 p1, vec2 p2, vec2 p3, float t) {
31     float u = 1.-t;
32     return u*u*u*p0 + 3.*u*u*t*p1 + 3.*u*t*t*p2 + t*t*t*p3;
33 }
34
35 void main() {
36     float pHash = hash(a_source);
37
38     // Dynamic Bezier control points from inter-slide spatial delta
39     vec2 delta = a_target - a_source;
40     vec2 cp1 = a_source + delta * u_cp1_coeff;
41     vec2 cp2 = a_target - delta * u_cp2_coeff;
42
43     // Trajectory with minimum-jerk easing + curl field perturbation
44     float t = ease(u_progress);
45     float E = sin(3.14159265 * t); // flight envelope
46     vec2 pos = cubicBezier(a_source, cp1, cp2, a_target, t)
47         + E * kernelField(a_source, t, u_seed);
48
49     // Heatmap density sampling at current normalised position
50     vec4 heat = texture(u_heatmap, pos);
51     float rho = a_semanticClass < 0.5 ? heat.r :
52         a_semanticClass < 1.5 ? heat.g : heat.b;
53     v_color = vec3(rho);
54
55     // Aura swarm (static state, non-structure particles only)
56     float depth = 1.0;
57     if (u_progress >= 1.0 && a_isStructure < 0.5) {
58         float r = mix(0.12, 0.03, rho); // wander radius
59         float f = fract(u_time*0.5 + pHash);
60         pos += vec2(sin(u_time*2.+pHash*10.),
61             cos(u_time*1.5+pHash*8.)) * r * (1.-f);
62         depth = 0.5 + 0.5*sin(u_time*3. + pHash*10.); // depth breathing
63     }
64
65     // NDC conversion with Y-flip (image top-down to OpenGL NDC)
66     pos.x = pos.x * 2. - 1.;
67     pos.y = -(pos.y * 2. - 1.);
68
69     // Aspect-ratio "contain" scaling (target: 16:9)
70     float A = 16./9.;
71     if (u_aspect > A) pos.x *= A/u_aspect;
72     else pos.y *= u_aspect/A;
73
74     gl_Position = vec4(pos, 0., 1.);
75     v_pos=pos; v_depth=depth; v_hash=pHash;

```

```

76     v_isStructure=a_isStructure; v_semanticClass=a_semanticClass;
77     v_weight=a_sourceWeight;
78
79     // Point size selection
80     if (u_progress >= 1.0)
81         gl_PointSize = a_semanticClass > 1.5
82             ? mix(3.,6.,a_sourceWeight)    // text: larger, sharper
83             : mix(2.,5.,a_sourceWeight);    // aura/halo
84     else
85         gl_PointSize = mix(2.,5.,E);        // in-flight: size by envelope
86 }

```

5.2 Particle Fragment Shader (GLSL ES 3.00)

Listing 2: VVV Particle Fragment Shader — full production source

```

1 #version 300 es
2 precision highp float;
3 in float v_weight, v_depth, v_hash, v_isStructure, v_semanticClass;
4 in vec2 v_pos;    in vec3 v_color;
5 out vec4 outColor;
6 uniform int u_isLine;
7 uniform float u_progress, u_time;
8
9 void main() {
10     if (u_isLine == 0) {
11         // --- Point Splat Mode ---
12         vec2 c = gl_PointCoord - .5;
13         float d = length(c);
14         float coreA;    vec3 coreC = vec3(0.);
15
16         if (u_progress >= 1.0) {
17             if (v_semanticClass > 1.5) {
18                 // CLASS 2 TEXT: always pure black, maximum sharpness
19                 coreC = vec3(0.);
20                 coreA = d > 0.4 ? 0. : 1.;
21             } else if (v_isStructure > 0.5) {
22                 coreC = vec3(0.);
23                 coreA = smoothstep(.13,.08,d)*(.8+v_weight*.2);
24             } else {
25                 coreC = mix(vec3(.4),vec3(.1),v_weight);
26                 coreA = smoothstep(.13,.08,d)*.4;
27             }
28         } else {
29             // In-flight: semi-transparent grey splat
30             coreC = mix(vec3(.4,.45,.5),vec3(.1,.1,.15),v_weight);
31             coreA = smoothstep(.4,.1,d)*(.6+v_weight*.4);
32         }
33
34         // --- Phantom Orbiting Particles (aura halo effect) ---
35         vec3 phantomC = mix(v_color,vec3(.4),.2);
36         float phantomA = 0.;
37         if (u_progress >= 1.0 && v_semanticClass < 1.5) {
38             for (int i=1; i<=3; i++) {
39                 float fi=float(i);
40                 float th=u_time*(0.8+fi*.4)+v_hash*6.28+fi*2.1;
41                 float r =0.12+0.18*sin(0.5*u_time+v_hash*10.+fi);
42                 vec2 q =c-vec2(cos(th),sin(th))*r;
43                 phantomA += smoothstep(.05,.02,length(q));
44             }
45             phantomA=min(phantomA,1.)*mix(.15,.45,v_depth);
46             if(v_isStructure>.5) phantomA*=.4;
47         }
48
49         // --- Composite ---
50         float wC=coreA, wP=phantomA;
51         vec3 fc=(coreC*wC+phantomC*wP)/(wC+wP+.0001);
52         float fa=min(coreA+phantomA,1.);
53         if(fa<0.01) discard;
54         outColor=vec4(fc,fa);
55     }

```



```

56     } else {
57         // --- Line Sweep Mode ---
58         float baseA=0.2;    vec3 lc=vec3(.5);
59         if (u_progress >= 1.0) {
60             float sw=fRACT(u_time*.1);
61             float dd=min(abs(v_hash-sw),1.-abs(v_hash-sw));
62             float al=smoothstep(.015,0.,dd); // animated sweep highlight
63             if(v_semanticClass>1.5)         baseA=0.; // text: no lines
64             else if(v_semanticClass>0.5) { baseA=mix(.05,.2,al); lc=vec3(.7); }
65             else if(v_isStructure>.5)      { baseA=.15+.1*v_weight; }
66             else                           { baseA=mix(.05,.35,al); }
67         }
68         outColor=vec4(lc,baseA);
69     }
70 }

```

Python Compiler — VVV.py

6.1 Importance Sampling Algorithm

Listing 3: Importance-sampling particle encoder (core)

```

1  import numpy as np
2  from PIL import Image, ImageOps, ImageFilter
3  import struct, base64, io
4
5  def encode_slide_image(image_path, n_particles=20_000):
6      img = Image.open(image_path).convert("RGB")
7      img = img.resize((1920, 1080), Image.Resampling.LANCZOS)
8      gray = ImageOps.grayscale(img)
9      gray_np = np.array(gray, dtype=np.float32)
10     edge_np = np.array(gray.filter(ImageFilter.FIND_EDGES),
11                        dtype=np.float32)
12
13     # Inverse luminance: dark pixels -> high sampling probability
14     inv_lum = 255.0 - gray_np
15     text_mask = np.clip(inv_lum - 5.0, 0.0, 255.0)
16
17     # Importance probability map:
18     # 4x weight on text/dark pixels
19     # 2x weight on edge pixels
20     # 0.5 epsilon floor (all pixels have nonzero probability)
21     prob_map = text_mask * 4.0 + edge_np * 2.0 + 0.5
22     prob_flat = prob_map.flatten() / prob_map.sum()
23
24     # Importance-sample N particle positions
25     w, h = 1920, 1080
26     indices = np.random.choice(w*h, size=n_particles,
27                               replace=True, p=prob_flat)
28     ys = (indices // w).astype(np.int32)
29     xs = (indices % w).astype(np.int32)
30
31     buf = bytearray(n_particles * 10)
32     for i in range(n_particles):
33         px, py = xs[i], ys[i]
34         lum, edg = inv_lum[py,px], edge_np[py,px]
35
36         # Top-down normalised position (shader inverts Y itself)
37         xu = max(0, min(65535, int(px/(w-1)*65535)))
38         yu = max(0, min(65535, int(py/(h-1)*65535)))
39         wt = max(15, min(255, int(lum/255*255)))
40
41         # Semantic class: text > edge > background
42         if lum > 80: radius, sem = 5, 255 # TEXT (always black)
43         elif edg > 30: radius, sem = 4, 127 # HALO
44         else: radius, sem = 3, 0 # BASE
45
46         o = i * 10

```

```

47     struct.pack_into("<HH", buf, o, xu, yu)
48     buf[o+4]=wt; buf[o+5]=radius; buf[o+6]=0; buf[o+7]=0
49     buf[o+8]=0;   buf[o+9]=sem
50
51     support_b64 = base64.b64encode(bytes(buf)).decode()
52
53     # Heatmap = actual slide image as PNG 512x512.
54     # CRITICAL: latent_b64 is rendered as visible background by quad
55     # renderer AND sampled as u_heatmap GPU texture. Must be the real
56     # slide image, not a synthetic mask.
57     hm = img.resize((512, 512), Image.Resampling.LANCZOS)
58     b = io.BytesIO()
59     hm.save(b, format="PNG", compress_level=6)
60     latent_b64 = base64.b64encode(b.getvalue()).decode()
61
62     return latent_b64, support_b64

```

Progressive Loading Architecture

A 20-slide presentation at 20,000 particles each embeds $20 \times 200,000 = 4,000,000$ bytes of raw binary in Base64 (≈ 5.3 MB for particle data alone, plus PNG heatmaps ≈ 12 MB total). Loading everything before rendering would cause unacceptable blank-screen delay.

3-Tier Delivery Strategy

Tier 1 — Instant Boot Slide #01 embedded in `window._INJECTED_MANIFEST`. Renders at 0 ms.

Tier 2 — Deferred Chunks Slides #2–#N in `<script type="application/vvv-chunk">` elements. Not parsed until read.

Tier 3 — Idle-Queue Decode JavaScript idle-queue decodes chunks with 120 ms gaps, pre-loading before user navigates.

CLI Reference

```

# Minimal -- auto-detects slides/ directory
python VVV.py

# Full flags
python VVV.py --slides ./slides --outdir ./build --out index.html

# Windows one-click
run_build.bat

```

Flag	Default	Description
<code>-slides</code>	<code>./slides</code>	Directory of PNG/JPG input slides
<code>-outdir</code>	<code>./build</code>	Output directory
<code>-out</code>	<code>index.html</code>	Output filename

Troubleshooting

Symptom	Root Cause & Fix
Background turns green	<code>latent_b64</code> is a synthetic heatmap. Fix: encode actual slide image.
Particles as noise	Binary format mismatch (40 bytes vs 10). Verify <code>stride=10</code> .
Text upside-down	Y inverted twice. Store top-down; let shader flip.
Wrong particle colours	Synthetic heatmap channels. Fix: use real slide as heatmap.
Old slide after build	Manifest injection failed. Check <code>_INJECTED_MANIFEST</code> JSON.
Low text density	Lower <code>inv_lum > 80</code> threshold or increase <code>n_particles</code> .

Key Lessons Learned

- Always read the vertex shader first.** The entire binary format is encoded in the `vertexAttribPointer` calls. Wrong assumption about `Float32` vs `UNSIGNED_SHORT` cost several major iterations.
- The `latent_b64` has two jobs.** It is simultaneously the visual background image (quad pass) and the `u_heatmap` GPU texture (vertex shader). Sending a synthetic mask breaks the visual background.
- Y-axis convention is a common bug source.** OpenGL NDC has $y = -1$ at bottom; image pixels have $y = 0$ at top. The vertex shader handles the flip; the encoder must not pre-flip.
- Importance sampling drives density quality.** At 20,000 particles, text is readable only if $> 80\%$ land on dark pixels. The map $p \propto 4 \text{inv_lum} + 2 \text{edges} + 0.5$ achieves this.
- The master file is ground truth.** When in doubt, decode a particle from `VVV_ideal.html` and compare field-by-field with encoder output.

Package Architecture & Metadata

File	Ext	Description
<code>VVV.html</code>	<code>.html</code>	Compiled standalone WebGL2 presentation (output)
<code>VVV_ideal.html</code>	<code>.html</code>	Master template with 20 pre-baked kourbanov.com slides
<code>VVV.py</code>	<code>.py</code>	CLI compiler and encoder
<code>VVV.md</code>	<code>.md</code>	Comprehensive documentation
<code>VVV.tex</code>	<code>.tex</code>	This document (LaTeX source)
<code>VVV.pdf</code>	<code>.pdf</code>	Compiled PDF of this document

Metric	Value
Particles per slide	20,000
Binary stride	10 bytes/particle
Heatmap resolution	512×512 PNG
Target frame rate	60 FPS
Build time (5 slides)	≈ 1.8 s
Output format	Single-file <code>.html</code>
Rendering engine	WebGL2 (GLSL ES 3.00)
Live deployment	kourbanov.com